

Question Index - All 112 Questions

Q#	Question
Q1	Linux boot process - BIOS/UEFI → bootloader → kernel → systemd
Q2	Linux filesystem hierarchy - /, /etc, /var, /opt, /tmp, /proc, /sys
Q3	File permissions - read, write, execute, chmod, chown, chgrp
Q4	Special permissions - SUID, SGID, sticky bit, umask
Q5	File types - regular, directory, symlink, hard link, device, socket, pipe
Q6	Users and groups - /etc/passwd, /etc/shadow, useradd, usermod
Q7	Process management - ps, top, htop, kill, nice, renice, nohup
Q8	Process states - running, sleeping, stopped, zombie, orphan
Q9	Package management - apt, yum/dnf, rpm, dpkg, snap
Q10	systemd - services, units, journalctl, systemctl
Q11	Cron jobs - scheduling, crontab syntax, /etc/cron.d, at command
Q12	Disk management - df, du, lsblk, fdisk, mount, fstab
Q13	Filesystem types - ext4, xfs, btrfs, tmpfs, NFS
Q14	I/O redirection - stdin, stdout, stderr, >, >>, 2>&1, pipes
Q15	Text processing - cat, head, tail, grep, wc, sort, uniq, cut
Q16	Find and locate - find by name, size, time, type, -exec
Q17	Archiving and compression - tar, gzip, bzip2, xz, zip

Q18	SSH - key-based auth, ssh-keygen, ssh-agent, config, SCP, rsync
Q19	Environment variables - export, PATH, .bashrc, .bash_profile
Q20	vi/vim basics - modes, navigation, editing, saving
Q21	Linux networking - ip, ifconfig, ping, traceroute, netstat, ss
Q22	DNS tools - nslookup, dig, /etc/resolv.conf, /etc/hosts
Q23	Firewall basics - iptables, ufw, firewalld
Q24	Log files - /var/log/syslog, auth.log, dmesg, journalctl
Q25	Screen and tmux - terminal multiplexers for persistent sessions
Q26	Special files - /dev/null, /dev/zero, /dev/urandom, /dev/stdin
Q27	Signals - SIGTERM, SIGKILL, SIGHUP, SIGUSR1, SIGINT
Q28	Man pages, help, whatis, apropos - navigating documentation
Q29	Isof, fuser, inotifywait - file monitoring and debugging
Q30	tee, watch, diff, comm - everyday utility commands
Q31	Shell scripting fundamentals - shebang, variables, quoting, exit codes
Q32	Conditionals - if/elif/else, test, [[]], comparisons
Q33	Loops - for, while, until, break, continue
Q34	Functions - declaration, arguments, return, local vars, getopts
Q35	Arrays and associative arrays in Bash
Q36	String manipulation - substring, replacement, length, splitting
Q37	jq for JSON processing - parsing API output and configs

Q38	cURL deep dive - headers, auth, debugging API calls
Q39	Command substitution, process substitution, subshells
Q40	grep, sed, awk - the text processing power trio
Q41	Regular expressions - basic vs extended, classes, quantifiers
Q42	xargs and parallel execution - processing argument lists
Q43	ACLs - setfacl, getfacl, when basic permissions aren't enough
Q44	trap and signal handling - cleanup on exit/error
Q45	Error handling - set -euo pipefail, error trapping, exit codes
Q46	Here documents, here strings, template generation
Q47	File descriptors, ulimits - too many open files, production tuning
Q48	fstab and mount options - noexec, nosuid, nodev, defaults
Q49	File locking and concurrent execution - flock
Q50	Process management from scripts - background, wait, job control
Q51	Bash debugging - set -x, bash -x, PS4, shellcheck
Q52	Backup strategies - rsync, dd, incremental tar, snapshots
Q53	Systemd targets (run levels) - multi-user, rescue, emergency
Q54	Network bonding/teaming - HA networking
Q55	Bash vs sh vs zsh vs dash - shell compatibility
Q56	LVM - physical volumes, volume groups, logical volumes
Q57	RAID levels - 0, 1, 5, 6, 10 - when to use which

Q58	Swap space - sizing, swapon/swapoff, swappiness tuning
Q59	Linux kernel - modules, sysctl, /proc tuning, kernel parameters
Q60	Linux capabilities - fine-grained privileges replacing root
Q61	Namespaces and cgroups - the foundation of containers
Q62	SELinux and AppArmor - mandatory access control
Q63	Linux networking deep dive - routing, iptables chains, NAT, bridging
Q64	TCP/IP stack tuning - socket buffers, conntrack, TIME_WAIT
Q65	Storage performance - I/O schedulers, fio, disk latency
Q66	Memory management - OOM killer, page cache, hugepages
Q67	Linux hardening - CIS benchmarks, automated scanning, lynis
Q68	/proc and /sys - runtime kernel inspection and tuning
Q69	Disk encryption - LUKS setup and key management
Q70	Systemd advanced - custom units, dependencies, resource limits
Q71	Infrastructure automation with shell scripts
Q72	Log aggregation and rotation - logrotate, rsyslog, journald
Q73	Security hardening - SSH hardening, fail2ban, audit framework
Q74	Certificate management - openssl, CSR, self-signed, Let's Encrypt
Q75	chroot - jail environments and their role before containers
Q76	NFS, SMB/CIFS - network file sharing
Q77	Linux in cloud - cloud-init, user-data, metadata service

Q78	Container runtime from scratch - unshare and nsenter
Q79	strace and ltrace - system call tracing
Q80	Bash vs Python for DevOps - when to switch languages
Q81	Server disk 100% full - find and fix without losing data
Q82	100% disk but du shows free space - deleted files held open
Q83	Cannot SSH into server - systematic debugging
Q84	Running out of inodes but disk not full
Q85	Process consuming 100% CPU - identify and resolve
Q86	OOM killer killed production process - diagnose and prevent
Q87	Zombie processes accumulating - find parent and fix
Q88	Cron job running but producing no output - debug
Q89	Script works manually but fails in cron - environment issues
Q90	Permission denied despite correct permissions - SELinux
Q91	DNS resolution suddenly stopped on a server
Q92	Server time drifting - NTP causing auth failures
Q93	/var/log filling up disk - log rotation broken
Q94	SSH key auth stopped working after server update
Q95	Filesystem went read-only - diagnose and recover
Q96	Network interface flapping - up and down repeatedly
Q97	Fork bomb executed - system unresponsive

Q98	Kernel panic - check and recover
Q99	Server compromised - incident response and forensics
Q100	Server suddenly very slow - systematic load diagnosis
Q101	High load average but CPU idle - I/O wait diagnosis
Q102	Can't delete file even as root - immutable flag
Q103	Application can't bind to port 80 - port conflict, capabilities
Q104	Script works on one server fails on another - shell differences
Q105	High TIME_WAIT connections - socket exhaustion
Q106	SSH connects but drops after authentication
Q107	Swap thrashing - system crawling despite available memory
Q108	Accidentally ran rm -rf on wrong directory - recovery
Q109	Accidentally ran chmod -R 777 / - recovery
Q110	Write a script: disk monitoring with alerting
Q111	Parse 50GB log file - top 10 IPs and attack patterns
Q112	Write a deployment script with rollback capability

SECTION 1 - Junior Core Fundamentals (Q1-Q30)

Q1 Linux boot process - BIOS/UEFI → bootloader → kernel → systemd

The Simple Answer

When you press the power button, the machine goes through: BIOS/UEFI firmware initializes hardware → bootloader (GRUB) loads the kernel → kernel initializes hardware drivers and mounts root filesystem → systemd (PID 1) starts all services in dependency order. Understanding this chain is critical because boot failures at each stage have different symptoms and fixes.

The Analogy - Starting a Restaurant

BIOS is like turning on electricity and gas in the building. GRUB is the restaurant manager deciding which menu (kernel) to use today. The kernel is the head chef checking all equipment in the kitchen. systemd is the staff arriving - waiters, cooks, dishwashers - each starting in the right order. You cannot seat customers (run applications) until the entire chain completes.

Each Stage in Detail

BIOS/UEFI - Tests hardware (POST), finds the boot device, loads the bootloader from the MBR or EFI partition. If this fails: no screen output at all, or hardware error beeps.

GRUB - Displays a menu if configured, loads the kernel and initramfs into memory. If this fails: GRUB rescue prompt, 'operating system not found.'

Kernel - Initializes CPU, memory, and hardware drivers. Mounts initramfs (temporary root), then switches to the real root filesystem. If this fails: kernel panic, hung boot with driver error messages.

systemd (init) - PID 1. Reads unit files, builds dependency graph, starts services in parallel. If this fails: boot hangs at a specific service, or drops to emergency shell. Debug with `systemctl --failed` and `journalctl -xb`.

Q2 Linux filesystem hierarchy - /, /etc, /var, /opt, /tmp, /proc, /sys

The Simple Answer

Linux organizes everything in a single tree starting from /. Unlike Windows where each drive has its own letter, Linux mounts all storage under one hierarchy. Every directory has a specific purpose defined by the Filesystem Hierarchy Standard (FHS).

Key Directories

/ - Root of everything. The top of the tree.

/etc - System configuration files. SSH config, hostname, DNS resolver, service configs. Think of it as the control panel.

/var - Variable data that changes during operation. Logs (/var/log), mail spools, package caches. This is where disk fills up most often.

/home - User home directories. Each user gets /home/username.

/tmp - Temporary files. Cleared on reboot. Anyone can write here (sticky bit set). Never store anything important here.

/opt - Optional/third-party software. Applications not managed by the package manager go here.

/proc - Virtual filesystem exposing kernel and process information. /proc/cpuinfo shows CPU details, /proc/meminfo shows memory. Files here do not exist on disk - they are generated by the kernel on demand.

/sys - Virtual filesystem for kernel device and driver information. Used for hardware configuration and tuning.

/usr - User programs and libraries. /usr/bin for binaries, /usr/lib for libraries, /usr/share for shared data.

/dev - Device files. Every hardware device is represented as a file. /dev/sda is the first disk, /dev/null is the bit bucket.

The interview question behind this: "Where would you look for the SSH configuration?" (/etc/ssh/). "Where are log files?" (/var/log/). "What fills up most?" (/var, especially /var/log). Knowing the hierarchy shows you navigate Linux systems confidently.

Q3 File permissions - read, write, execute, chmod, chown, chgrp

The Simple Answer

Every file and directory has three permission sets: owner, group, and others. Each set can have read (r=4), write (w=2), and execute (x=1). Permissions are displayed as rwxrwxrwx - three groups of three. chmod changes permissions, chown changes ownership, chgrp changes group.

The Analogy - Office Document Access

Think of a shared office document. The owner (author) has full control. The team (group) might have read and write access. Everyone else (others) might only read. chmod is like changing the sharing settings. chown is like transferring document ownership. chgrp is like moving the document to a different team's folder.

Numeric vs Symbolic

Numeric: chmod 755 file - Owner: rwx(7), Group: r-x(5), Others: r-x(5). **Symbolic:** chmod u+x file - Add execute for user. chmod go-w file - Remove write for group and others.

Common Permission Patterns

755 - Directories and executables. Owner can do everything, others can read and traverse.

644 - Regular files. Owner reads and writes, others read only.

600 - Private files (SSH keys, config with secrets). Only owner can read and write.

700 - Private directories. Only owner can access.

The #1 permission mistake: Setting 777 (everything open to everyone) to "fix" a permission denied error. This is a security disaster. Always find the correct owner/group/permission rather than opening everything up.

Q4 Special permissions - SUID, SGID, sticky bit, umask

SUID (Set User ID)

When SUID is set on an executable, it runs with the file owner's permissions regardless of who executes it. The classic example: `/usr/bin/passwd` is owned by root with SUID set. Any user can run it and it has root permissions to update `/etc/shadow`. Without SUID, normal users could not change their passwords.

SGID (Set Group ID)

On a file: runs with the file's group permissions. On a directory: new files created inside inherit the directory's group instead of the creator's primary group. Essential for shared team directories where all files should belong to the team group.

Sticky Bit

On a directory: only the file owner can delete their files, even if others have write permission on the directory. `/tmp` has the sticky bit - anyone can create files there, but you can only delete your own files.

umask

umask sets the default permissions for newly created files and directories. A umask of 022 means new files get 644 (666-022) and new directories get 755 (777-022). The umask subtracts permissions from the maximum default. Check with umask command, set in `.bashrc` or `/etc/profile`.

Interview question: "A new file is created with permissions 644 but you need it to be 600 by default. How?" Change the umask from 022 to 077. umask 077 means new files get 600 (666-077) and directories get 700 (777-077).

Q5 File types - regular, directory, symlink, hard link, device, socket, pipe

The Simple Answer

Linux has seven file types, each indicated by the first character in `ls -l` output.

- **(regular file)** - Text files, binaries, images. The most common type.

d (directory) - Contains other files and directories.

l (symbolic link) - A pointer to another file. Like a shortcut. Can point across filesystems. If the target is deleted, the symlink is broken (dangling).

Hard link - A second name for the same file data (same inode). Unlike symlinks, hard links cannot cross filesystems and cannot point to directories. Deleting one hard link does not affect others - the data is removed only when the last hard link is deleted.

b (block device) - Disk drives (/dev/sda). Data is read in blocks.

c (character device) - Terminals, serial ports. Data is read character by character.

s (socket) - Inter-process communication endpoint. MySQL uses /var/run/mysqld/mysqld.sock.

p (named pipe/FIFO) - Inter-process communication pipe with a name in the filesystem.

The interview question: "What is the difference between a hard link and a symbolic link?" Hard links share the same inode - they ARE the same file with two names. Symlinks are separate files pointing to a path. Hard links survive target rename. Symlinks break on target delete. Hard links cannot cross filesystems. Symlinks can.

Q6 Users and groups - /etc/passwd, /etc/shadow, useradd, usermod

The Simple Answer

Linux is a multi-user operating system. Users are defined in /etc/passwd (username, UID, home directory, shell). Passwords are hashed in /etc/shadow (only readable by root). Groups are defined in /etc/group. Every user has a primary group and can belong to multiple secondary groups.

Key Commands

useradd - Create a user. `useradd -m -s /bin/bash -G docker,sudo devuser` creates a user with a home directory, bash shell, and membership in docker and sudo groups.

usermod - Modify existing user. `usermod -aG docker devuser` adds docker group. The `-a` flag is critical - without it, the user is removed from all other secondary groups.

passwd - Set or change a user's password.

id - Show a user's UID, GID, and group memberships. Essential for debugging permission issues.

The gotcha: `usermod -G docker devuser` (without `-a`) replaces ALL secondary groups with just docker. The user loses sudo access. Always use `-aG`. This is one of the most common sysadmin mistakes.

Q7 Process management - ps, top, htop, kill, nice, renice, nohup

The Simple Answer

Every running program is a process with a unique PID. Linux provides tools to list, monitor, signal, and prioritize processes.

Key Commands

ps aux - Snapshot of all processes. Shows PID, CPU%, MEM%, command. `ps aux | grep nginx` finds nginx processes.

top - Real-time process monitor. Shows CPU, memory, load average. Press M to sort by memory, P by CPU, k to kill a process.

htop - Enhanced top with color, scrolling, tree view, and mouse support. Preferred for interactive monitoring.

kill - Send a signal to a process. `kill PID` sends SIGTERM (graceful stop). `kill -9 PID` sends SIGKILL (force kill). Always try SIGTERM first - SIGKILL does not allow cleanup.

nice - Start a process with a specific priority. `nice -n 10 ./heavy_job` starts with lower priority. Range: -20 (highest) to 19 (lowest).

renice - Change priority of a running process. `renice -n 5 -p 1234` lowers priority of PID 1234.

nohup - Run a command that survives terminal logout. `nohup ./script.sh &` runs in the background and survives disconnection. Output goes to `nohup.out`.

Q8 Process states - running, sleeping, stopped, zombie, orphan

The Simple Answer

R (Running) - Actively using CPU or ready to run.

S (Sleeping) - Waiting for an event (I/O, timer, signal). Most processes are in this state most of the time.

D (Uninterruptible Sleep) - Waiting for I/O that cannot be interrupted. Cannot be killed. Common cause of high load average with low CPU usage.

T (Stopped) - Paused by a signal (Ctrl+Z) or debugger. Can be resumed with `fg` or `bg`.

Z (Zombie) - Process finished but its parent has not read its exit status. Takes up a process table entry but no resources. Caused by buggy parent processes that do not call `wait()`.

Orphan - Parent died before the child. Adopted by PID 1 (systemd), which will eventually reap it. Not a problem - systemd handles it.

Why zombies matter in interviews: "What is a zombie process and how do you fix it?" A zombie is a dead process whose parent never called `wait()`. You cannot kill a zombie - it is already dead. You must fix or restart the parent process. If the parent is killed, `init/systemd` adopts the zombie and reaps it.

Q9 Package management - apt, yum/dnf, rpm, dpkg, snap

The Simple Answer

Package managers install, update, and remove software and handle dependencies automatically.

Debian/Ubuntu family: dpkg handles individual .deb files. apt manages repositories and dependencies. apt update refreshes the package list. apt install nginx installs with all dependencies. apt upgrade updates all packages.

Red Hat/CentOS/Fedora family: rpm handles individual .rpm files. yum (older) or dnf (newer) manages repositories and dependencies. dnf install nginx installs with dependencies.

snap - Universal packages that work across distributions. Self-contained with all dependencies bundled. Larger than native packages but more portable.

The command every DevOps engineer should know: apt-get update && apt-get install -y package && rm -rf /var/lib/apt/lists/* - update, install, and clean up in one line. Essential for Dockerfiles where layer size matters.

Q10 systemd - services, units, journalctl, systemctl

The Simple Answer

systemd is the init system and service manager on modern Linux distributions. It starts and manages all system services, handles dependencies, provides logging through journald, and replaces older init systems (SysVinit, Upstart).

Essential Commands

systemctl start nginx - Start a service. **systemctl stop nginx** - Stop a service. **systemctl restart nginx** - Stop then start. **systemctl reload nginx** - Reload config without stopping. **systemctl enable nginx** - Start automatically on boot. **systemctl disable nginx** - Do not start on boot. **systemctl status nginx** - Show current state, last logs, PID. **systemctl list-units --failed** - Show failed services.

journalctl - The Log Viewer

journalctl -u nginx - Logs for a specific service. **journalctl -u nginx --since "1 hour ago"** - Recent logs. **journalctl -f** - Follow all logs in real time (like tail -f). **journalctl -b** - Logs from current boot. **journalctl -p err** - Only error-level messages.

The debug workflow: Service not working? systemctl status service shows current state. journalctl -u service -n 50 shows recent logs. journalctl -u service -p err shows only errors. This three-command sequence diagnoses most service issues.

Q11 Cron jobs - scheduling, crontab syntax, /etc/cron.d

The Simple Answer

Cron executes commands on a schedule. The crontab file defines when and what to run. Five time fields: minute (0-59), hour (0-23), day of month (1-31), month (1-12), day of week (0-7).

Common Patterns

0 2 * * * - Daily at 2:00 AM. ***/5 * * * *** - Every 5 minutes. **0 0 * * 0** - Weekly on Sunday at midnight. **0 9 1 * *** - Monthly on the 1st at 9 AM.

Key Commands

crontab -e - Edit current user's crontab. **crontab -l** - List current user's crontab. **crontab -r** - Remove entire crontab (dangerous!).

Where Cron Jobs Live

/var/spool/cron/crontabs/ - Per-user crontabs. **/etc/crontab** - System-wide crontab with username field. **/etc/cron.d/** - Drop-in directory for package-installed cron jobs. **/etc/cron.daily/**, **cron.hourly/**, **cron.weekly/** - Scripts placed here run at those intervals via anacron.

The #1 cron debugging issue: The cron environment is minimal - PATH is limited, HOME may not be set, no .bashrc is sourced. Scripts that work in your terminal fail in cron because they rely on environment variables that do not exist. Always use full paths in cron scripts: /usr/bin/python3 instead of python3.

Q12 Disk management - df, du, lsblk, fdisk, mount, fstab

The Simple Answer

df -h - Shows disk space usage per filesystem. The first thing to check when disk is full.

du -sh /path - Shows disk usage of a directory. **du -sh /*** shows which top-level directory is consuming the most space.

lsblk - Lists all block devices (disks, partitions) in a tree view. Shows device names, sizes, mount points.

fdisk - Partition table editor. **fdisk -l** lists all partitions. Used for creating and managing disk partitions.

mount - Attaches a filesystem to the directory tree. **mount /dev/sdb1 /mnt/data** makes the partition accessible at /mnt/data.

fstab - /etc/fstab defines filesystems to mount automatically at boot. Each line specifies device, mount point, filesystem type, and mount options.

Q13 Filesystem types - ext4, xfs, btrfs, tmpfs, NFS

The Simple Answer

ext4 - The default on most Linux distributions. Mature, reliable, well-understood. Supports filesystems up to 1 EB and files up to 16 TB. Good for general purpose.

xfs - Default on RHEL/CentOS. Excellent for large files and high I/O workloads. Cannot be shrunk (only grown). Good for databases and media storage.

btrfs - Advanced features: snapshots, compression, checksums, subvolumes. Good for flexible storage management. Less mature than ext4/xfs.

tmpfs - Filesystem stored in RAM. Extremely fast. Contents lost on reboot. Used for /tmp and /dev/shm.

NFS - Network File System. Shares directories across machines over the network. Transparent to applications - they access NFS mounts like local directories.

Q14 I/O redirection - stdin, stdout, stderr, pipes

The Simple Answer

Every process has three standard streams: stdin (0), stdout (1), stderr (2). Redirection controls where these streams go.

> - Redirect stdout to file (overwrite). **ls > filelist.txt.** **>>** - Redirect stdout to file (append). **2>** - Redirect stderr to file. **command 2> errors.log.** **2>&1** - Redirect stderr to wherever stdout goes. **command > all.log 2>&1** captures both. **|** - Pipe stdout of one command to stdin of another. **cat file | grep error | wc -l.**

The Order Matters

command > file 2>&1 - Correct. Sends stdout to file, then sends stderr to same file. **command 2>&1 > file** - Wrong! Sends stderr to terminal (where stdout was), then sends stdout to file. Stderr still goes to terminal.

/dev/null - The bit bucket. Discard output: **command > /dev/null 2>&1** silences everything. In modern bash: **command &>/dev/null** is equivalent.

Q15 Text processing - cat, head, tail, grep, wc, sort, uniq, cut

Essential Commands

cat - Display file contents. **cat -n** adds line numbers. **head -20 file** - First 20 lines. **tail -20 file** - Last 20 lines. **tail -f file** - Follow file in real time (logs). **grep pattern file** - Search for pattern. **-i** case-insensitive, **-r** recursive, **-v** invert match, **-c** count matches. **wc** - Count lines (**-l**), words (**-w**), characters (**-c**). **sort** - Sort lines. **-n** numeric, **-r** reverse, **-k2** by second field. **uniq** - Remove adjacent duplicates. Almost always used after sort: **sort file | uniq -c** counts occurrences. **cut** - Extract fields. **cut -d: -f1 /etc/passwd** extracts usernames (first field, colon-delimited).

The Power Pipeline

cat access.log | grep "POST /api" | cut -d' ' -f1 | sort | uniq -c | sort -rn | head -10 - Find the top 10 IP addresses making POST requests. This pipeline pattern appears in almost every Linux interview.

Q16 Find and locate - find by name, size, time, type, -exec

The Simple Answer

find - Searches the filesystem in real time. Flexible but slow on large filesystems.

locate - Searches a pre-built database. Instant results but database may be stale. Update with updatedb.

find Examples

find / -name "*.log" -size +100M - Files named .log larger than 100 MB. **find /var -type f -mtime +30 -delete** - Delete files not modified in 30 days. **find . -name "*.py" -exec grep "import os" {} \;** - Search inside Python files. **find / -perm -4000** - Find all SUID files (security audit).

Q17 Archiving and compression - tar, gzip, bzip2, xz, zip

The Simple Answer

tar - Bundles files into a single archive. Does not compress by itself. **tar -cf archive.tar /dir** creates. **tar -xf archive.tar** extracts.

gzip - Fast compression. **tar -czf archive.tar.gz /dir**. **bzip2** - Better compression, slower. **tar -cjf archive.tar.bz2 /dir**. **xz** - Best compression, slowest. **tar -cJf archive.tar.xz /dir**.

The command every engineer types from memory: **tar -xzf filename.tar.gz** - Extract a gzipped tarball. The flags: x=extract, z=gzip, f=filename.

Q18 SSH - key-based auth, ssh-keygen, ssh-agent, config, SCP, rsync

The Simple Answer

SSH provides encrypted remote access. Key-based authentication is more secure than passwords - a private key stays on your machine, the public key goes on the server.

Key Generation and Setup

ssh-keygen -t ed25519 generates a key pair. **ssh-copy-id user@server** installs the public key on the server. After this, **ssh user@server** connects without a password.

SSH Config File (~/.ssh/config)

Define hosts with aliases: **Host prod, HostName 10.0.1.5, User deploy, IdentityFile ~/.ssh/prod_key**. Then just **ssh prod** connects with all settings applied.

File Transfer

SCP - Simple copy. **scp file user@server:/path/**. **rsync** - Incremental sync. **rsync -avz --progress /local/ user@server:/remote/**. Only transfers changed bytes. Resumable. Far superior to SCP for large transfers.

Security: Always disable password authentication on production servers (PasswordAuthentication no in /etc/ssh/sshd_config). Only allow key-based auth. Use ed25519 keys (more secure than RSA). Never share private keys.

Q19 Environment variables - export, PATH, .bashrc, .bash_profile

The Simple Answer

Environment variables configure the shell and programs. PATH tells the shell where to find executables. HOME is the user's home directory. LANG sets the locale.

export

Variables set without export are local to the current shell. export makes them available to child processes. export MY_VAR="value" - children inherit it. MY_VAR="value" - only this shell sees it.

Initialization Files

.bash_profile - Runs on login shells (SSH, console login). Sets PATH and environment variables. **.bashrc** - Runs on non-login interactive shells (opening a terminal). Sets aliases and functions. **/etc/environment** - System-wide variables for all users. **/etc/profile.d/*.sh** - System-wide scripts run on login.

The common confusion: "I added something to .bashrc but it does not work when I SSH in." SSH creates a login shell, which reads .bash_profile, not .bashrc. The fix: source .bashrc from .bash_profile.

Q20 vi/vim basics - modes, navigation, editing, saving

The Simple Answer

vi/vim is available on every Linux system. Even if you prefer nano, knowing vim basics is essential because you will encounter systems where it is the only editor available.

Three Modes

Normal mode - Navigate and execute commands. Default when vim opens. **Insert mode** - Type text. Press i to enter, Esc to exit. **Command mode** - Execute commands. Press : from normal mode.

Essential Commands

:w - Save. **:q** - Quit. **:wq** - Save and quit. **:q!** - Quit without saving. **dd** - Delete a line. **yy** - Copy a line. **p** - Paste. **/pattern** - Search. **:%s/old/new/g** - Replace all occurrences.

Q21 Linux networking - ip, ifconfig, ping, traceroute, netstat, ss

The Simple Answer

ip addr - Show all network interfaces and IP addresses (replaces ifconfig). **ip route** - Show routing table. **ping host** - Test connectivity. **traceroute host** - Show the network path. **ss -tlnp** - Show listening TCP ports with process names (replaces netstat). **netstat -tlnp** - Older equivalent, may not be installed.

ss vs netstat

ss is faster and provides more information. ss -tlnp shows listening TCP ports. ss -tunap shows all connections (TCP+UDP). ss -s shows socket statistics summary.

Q22 DNS tools - nslookup, dig, /etc/resolv.conf, /etc/hosts

The Simple Answer

dig domain - Query DNS. Shows the full answer including TTL and authoritative server. The preferred DNS debugging tool. **nslookup domain** - Simpler DNS query. Less detail than dig. **/etc/resolv.conf** - Configures which DNS servers the system uses. nameserver 8.8.8.8 entries. **/etc/hosts** - Local hostname mapping. Checked before DNS. Used for overrides and local development.

DNS Resolution Order

Linux checks /etc/hosts first, then DNS (configured in /etc/resolv.conf). The order is defined in /etc/nsswitch.conf (hosts: files dns). If an entry exists in /etc/hosts, DNS is never queried for that name.

Q23 Firewall basics - iptables, ufw, firewalld

The Simple Answer

iptables - Low-level. Defines rules in chains (INPUT, OUTPUT, FORWARD). Complex syntax. Direct kernel interface. Still used in many production systems and containers.

ufw (Uncomplicated Firewall) - Simplified frontend for iptables on Ubuntu. ufw allow 22 opens SSH. ufw enable activates. Much easier for basic rules.

firewalld - Dynamic firewall on RHEL/CentOS. Uses zones and services. firewall-cmd --add-service=http --permanent adds HTTP access.

Q24 Log files - /var/log/syslog, auth.log, dmesg, journalctl

The Simple Answer

/var/log/syslog (Debian) or **/var/log/messages** (RHEL) - General system logs. **/var/log/auth.log** or **/var/log/secure** - Authentication logs. SSH logins, sudo usage, failed passwords. **/var/log/kern.log** - Kernel messages. **dmesg** - Kernel ring buffer. Hardware detection, driver messages. Useful for debugging hardware issues.

journalctl

On systemd systems, journalctl is the primary log viewer. journalctl -u service filters by service. journalctl -p err shows errors only. journalctl --since "2025-01-01" filters by time.

Q25 Screen and tmux - terminal multiplexers

The Simple Answer

Terminal multiplexers let you run persistent terminal sessions that survive disconnection. You can SSH into a server, start a tmux session, run a long process, disconnect, reconnect later, and the process is still running in the same session.

tmux Essentials

tmux new -s mysession - Create named session. **tmux attach -t mysession** - Reattach. **Ctrl+b d** - Detach (session keeps running). **Ctrl+b c** - New window. **Ctrl+b %** - Split pane vertically. **Ctrl+b "** - Split horizontally.

Why DevOps engineers use tmux daily: Running a deployment that takes 30 minutes? Start it in tmux. If your SSH connection drops, the deployment continues. Reattach and check the result. Without tmux, a dropped connection kills the deployment mid-way.

Q26 Special files - /dev/null, /dev/zero, /dev/urandom

The Simple Answer

/dev/null - Discards everything written to it. Read from it: returns nothing. The bit bucket. command `> /dev/null 2>&1` silences all output.

/dev/zero - Infinite stream of null bytes. Used for creating files of specific sizes: `dd if=/dev/zero of=testfile bs=1M count=100` creates a 100 MB file.

/dev/urandom - Infinite stream of random bytes. Used for generating passwords, tokens, encryption keys. `head -c 32 /dev/urandom | base64` generates a random 32-byte base64 string.

/dev/stdin, /dev/stdout, /dev/stderr - File descriptors as paths. Some programs that only accept filenames can read from stdin via `/dev/stdin`.

Q27 Signals - SIGTERM, SIGKILL, SIGHUP, SIGUSR1, SIGINT

The Simple Answer

Signals are software interrupts sent to processes. Each signal has a default action but most can be caught and handled by the application.

SIGTERM (15) - Polite termination request. The process can catch it, clean up, and exit gracefully. kill PID sends this by default.

SIGKILL (9) - Force kill. Cannot be caught, blocked, or ignored. The kernel terminates the process immediately. No cleanup. kill -9 PID. Use only as last resort.

SIGINT (2) - Interrupt. Sent by Ctrl+C. Usually causes the process to exit.

SIGHUP (1) - Hangup. Originally meant terminal disconnected. Many daemons reload config on SIGHUP: kill -HUP \$(cat /var/run/nginx.pid) reloads nginx configuration.

SIGUSR1/SIGUSR2 - User-defined signals. Applications can define their own behavior. Some use SIGUSR1 to trigger log rotation or debug output.

SIGSTOP/SIGCONT - Pause and resume. Ctrl+Z sends SIGSTOP. fg or bg sends SIGCONT.

The production rule: Always kill with SIGTERM first. Wait 10-30 seconds. If the process does not exit, then use SIGKILL. SIGKILL prevents cleanup - database connections stay open, files are not flushed, locks are not released. Docker's stop command follows this exact pattern: SIGTERM → wait → SIGKILL.

Q28 Man pages, help, whatis, apropos

The Simple Answer

man command - Full manual page. man grep shows grep documentation with all flags. Navigate with arrow keys, search with /pattern, quit with q.

command --help - Quick usage summary. Faster than man for checking a specific flag.

whatis command - One-line description. whatis grep says "print lines that match patterns."

apropos keyword - Search all man pages for a keyword. apropos "network" lists all commands related to networking. Essential when you know what you want to do but not which command does it.

Q29 lsof, fuser, inotifywait - file monitoring and debugging

The Simple Answer

lsof - Lists open files. Since everything in Linux is a file (including network connections), lsof is the universal debugging tool.

lsof -i :8080 - Which process is using port 8080? lsof /var/log/syslog - Which processes have this file open? lsof -u username - All files opened by a user. lsof +D /tmp - All open files in /tmp directory.

fuser - Identifies processes using a file or filesystem. fuser -m /mnt/data - Who is using this mount? Essential before unmounting. fuser -k /mnt/data kills those processes.

inotifywait - Watches for filesystem events. inotifywait -m /etc/nginx/ watches for config changes. Used in scripts to trigger actions when files change - like reloading a service when its config is updated.

The debugging pattern: "Something is using port 8080 and I don't know what." `lsof -i :8080` shows the process name and PID. kill the process or reconfigure it. This scenario appears in almost every Linux interview.

Q30 tee, watch, diff, comm - everyday utilities

The Simple Answer

tee - Writes to both stdout and a file simultaneously. `command | tee output.log` shows output on screen AND saves it to a file. Use `tee -a` to append instead of overwrite.

watch - Runs a command repeatedly and shows the output. `watch -n 2 'df -h'` refreshes disk usage every 2 seconds. `watch -d` highlights changes between refreshes. Essential for monitoring something in real time.

diff - Compares two files line by line. `diff file1 file2` shows differences. `diff -u` gives unified format (the format used in patches and Git diffs).

comm - Compares two sorted files. Shows lines unique to each file and lines in common. Useful for comparing lists - finding users who exist in one system but not another.

SECTION 2 - Mid-Level Shell Scripting & System Administration (Q31-Q58)

Q31 Shell scripting fundamentals - shebang, variables, quoting, exit codes

The Simple Answer

Every shell script starts with a shebang (`#!/`) specifying the interpreter: `#!/bin/bash` or `#!/usr/bin/env bash` (more portable). Variables are assigned without spaces: `NAME="value"`. Quoting controls expansion: double quotes allow variable expansion, single quotes are literal. Exit codes indicate success (0) or failure (non-zero).

Variable Quoting - The #1 Source of Bugs

"\$VAR" - Double quotes: variable expands but spaces are preserved. Always use this. **'\$VAR'** - Single quotes: literal string, no expansion. Prints `$VAR` literally. **\$VAR** - Unquoted: word splitting and globbing happen. If `VAR` contains spaces or wildcards, it breaks. NEVER use unquoted variables.

Exit Codes

Every command returns an exit code. 0 = success, non-zero = failure. `$?` holds the last command's exit code. In scripts: exit 0 for success, exit 1 for failure. Callers check exit codes to decide what to do next.

The rule that prevents 90% of shell script bugs: Always quote your variables. `"$variable"` not `$variable`. Unquoted variables with spaces cause mysterious failures. `file="my file.txt"; rm $file` deletes "my" and "file.txt" - two files instead of one.

Q32 Conditionals - if/elif/else, test, [[]], comparisons

The Simple Answer

`if`, `elif`, `else` control flow based on conditions. `test` or `[ ]` evaluates conditions. `[[ ]]` is the Bash-specific enhanced version with better syntax.

String Comparisons

`[[ "$a" == "$b" ]]` - Equal. `[[ "$a" != "$b" ]]` - Not equal. `[[ -z "$a" ]]` - Empty string. `[[ -n "$a" ]]` - Non-empty.

Numeric Comparisons

`[[ "$a" -eq "$b" ]]` - Equal. `-ne` (not equal), `-lt` (less than), `-gt` (greater than), `-le` (less/equal), `-ge` (greater/equal).

File Tests

`[[ -f "$file" ]]` - Exists and is a regular file. `[[ -d "$dir" ]]` - Exists and is a directory. `[[ -r "$file" ]]` - Readable. `[[ -w "$file" ]]` - Writable. `[[ -x "$file" ]]` - Executable. `[[ -s "$file" ]]` - Exists and is not empty.

[[]] vs []

Always use `[[ ]]` in Bash scripts. It handles unquoted variables safely, supports `&&` and `||` directly, and supports pattern matching with `==`. The single bracket `[ ]` is POSIX-compatible but more error-prone.

Q33 Loops - for, while, until, break, continue

for Loop Patterns

Iterating a list: `for item in file1 file2 file3; do echo "$item"; done`. **Iterating command output:** `for file in $(find . -name "*.log"); do rm "$file"; done`. **C-style:** `for ((i=0; i<10; i++)); do echo $i; done`. **Iterating lines of a file:** `while IFS= read -r line; do echo "$line"; done < file.txt` - This is the correct way. Never use `for` line in `$(cat file)` - it breaks on spaces.

while Loop

`while [[ condition ]]; do ... done`. Common pattern: `while true; do ... sleep 5; done` for polling loops. `while read` processes input line by line - essential for parsing files and command output.

Loop Control

break - Exit the loop immediately. **continue** - Skip to the next iteration.

Q34 Functions - declaration, arguments, return, local vars, getopt

Function Basics

Declare with `function_name() { ... }` or `function function_name { ... }`. Arguments accessed as `$1`, `$2`, etc. `$@` is all arguments. `$#` is argument count. Return value is an exit code (0-255), not a string - use `echo` for string returns.

Local Variables

Variables in functions are global by default. Use `local` to scope them: `local result="value"`. Without `local`, a variable set inside a function overwrites the global variable - causing subtle bugs in larger scripts.

getopts - Argument Parsing

`getopts` parses command-line flags: `while getopts "hvf:" opt; do ...`. This handles `-h` (help), `-v` (verbose), `-f` filename (requires argument). The colon after `f` means `-f` takes an argument. Every production script should accept `-h` for help and parse arguments properly with `getopts` instead of positional parameters.

Q35 Arrays and associative arrays in Bash

Indexed Arrays

Declare: `arr=(one two three)`. Access: `${arr[0]}`. All elements: `${arr[@]}`. Length: `${#arr[@]}`. Append: `arr+=(four)`. Iterate: `for item in "${arr[@]}"; do echo "$item"; done`.

Associative Arrays (Bash 4+)

Declare: `declare -A map`. Set: `map[key]="value"`. Access: `${map[key]}`. All keys: `${!map[@]}`. Used for lookup tables, configuration parsing, and counting occurrences.

Always quote array expansions: `"${arr[@]}"` not `${arr[@]}`. Unquoted expansions break on elements containing spaces.

Q36 String manipulation - substring, replacement, length, splitting

String Operations in Bash

Length: `${#string}`. **Substring:** `${string:start:length}`. **Remove prefix:** `${string#pattern}` (shortest), `${string##pattern}` (longest). **Remove suffix:** `${string%pattern}` (shortest), `${string%%pattern}` (longest). **Replace first:** `${string/old/new}`. **Replace all:** `${string//old/new}`. **Uppercase:** `${string^^}`. **Lowercase:** `${string,,}`.

Practical Example

Extract filename from path: `path="/var/log/nginx/access.log"; filename=${path##*/}` gives "access.log".
Remove extension: `name=${filename%.*}` gives "access".

Q37 jq for JSON processing - parsing API output and configs

The Simple Answer

jq is the command-line JSON processor. Essential for parsing cloud CLI output, API responses, and configuration files. Without jq, you end up using grep and sed on JSON - fragile and error-prone.

Essential jq Patterns

Extract a field: `echo '{"name":"nginx","port":80}' | jq '.name'` outputs "nginx". **Without quotes:** `jq -r '.name'` outputs nginx (raw, no quotes). **Array iteration:** `jq '.items[]'` iterates array elements. **Filter:** `jq '.items[] | select(.status == "Running")'` filters by condition. **Multiple fields:** `jq '{name: .metadata.name, status: .status.phase}'` extracts and reshapes.

Real-World DevOps Usage

`aws ec2 describe-instances | jq -r '.Reservations[].Instances[] | select(.State.Name == "running") | .InstanceId'` - List running EC2 instance IDs.
`kubectl get pods -o json | jq -r '.items[] | select(.status.phase != "Running") | .metadata.name'` - List non-running Pods.

Q38 cURL deep dive - headers, auth, debugging API calls

The Simple Answer

cURL is the universal HTTP client for the command line. Every DevOps engineer uses it daily for testing APIs, checking endpoints, and debugging connectivity.

Essential Flags

-v - Verbose. Shows headers, TLS handshake, request/response details. The first flag to add when debugging. **-o file** - Save output to file. **-s** - Silent. No progress bar. **-f** - Fail silently on HTTP errors (return non-zero exit code). **-H "Header: Value"** - Add custom header. **-d '{"key": "value"}'** - Send request body (POST). **-X POST** - Set HTTP method. **-u user:pass** - Basic authentication. **-k** - Skip TLS verification (testing only, never production). **-w "%{http_code}"** - Print HTTP status code. **-L** - Follow redirects. **--connect-timeout 5** - Connection timeout.

Common Patterns

Health check: `curl -sf http://localhost:8080/health | | echo "DOWN"`. API call with auth: `curl -s -H "Authorization: Bearer $TOKEN" https://api.example.com/data | jq '.'`. Upload file: `curl -F "file=@report.pdf" https://upload.example.com/`.

Q39 Command substitution, process substitution, subshells

Command Substitution

`$(command)` captures the output of a command as a string. `result=$(date +%Y-%m-%d)` sets result to "2025-06-15". The older backtick syntax ``command`` works but is harder to nest.

Process Substitution

`<(command)` creates a temporary file-like object from command output. `diff <(sort file1) <(sort file2)` compares sorted versions without creating temp files. Only available in Bash, not POSIX sh.

Subshells

`(commands)` runs commands in a subshell. Variable changes inside do not affect the parent. Useful for temporary directory changes: `(cd /tmp && do_work)` - after the subshell, you are back in the original directory.

Q40 grep, sed, awk - the text processing power trio

grep - Search

`grep -r "ERROR" /var/log/` - Recursive search. `grep -i "error"` - Case-insensitive. `grep -v "DEBUG"` - Exclude lines. `grep -c "pattern"` - Count matches. `grep -A5 "FATAL"` - Show 5 lines after match. `grep -E "err|warn"` - Extended regex (egrep).

sed - Transform

sed 's/old/new/g' file - Replace all occurrences. sed -i 's/old/new/g' file - Edit in-place. sed '/pattern/d' file - Delete matching lines. sed -n '10,20p' file - Print lines 10-20.

awk - Extract and Compute

awk '{print \$1, \$3}' file - Print 1st and 3rd columns (space-delimited). awk -F: '{print \$1}' /etc/passwd - Custom delimiter (colon). awk '\$3 > 100 {print \$1}' - Conditional: print column 1 where column 3 exceeds 100. awk '{sum+=\$5} END {print sum}' - Sum column 5.

The text processing hierarchy: Use grep when you only need to search. Use sed when you need simple substitution. Use awk when you need column extraction, computation, or complex logic. Use Python when awk gets too complex.

Q41 Regular expressions - basic vs extended, classes, quantifiers

Basic Regex (BRE)

Used by grep (default), sed. Special characters: . (any char), * (zero or more of previous), ^ (start of line), \$ (end of line), [] (character class), \ (\) (grouping).

Extended Regex (ERE)

Used by grep -E, awk, sed -E. Adds: + (one or more), ? (zero or one), | (alternation), () (grouping without backslash).

Common Patterns

^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3} - IP address (approximate). ^#|^\$ - Comment or empty line. [0-9]{4}-[0-9]{2}-[0-9]{2} - Date format YYYY-MM-DD.

Q42 xargs and parallel execution

xargs

Converts stdin into arguments for a command. find . -name "*.tmp" | xargs rm - Delete all .tmp files. echo "a b c" | xargs -n1 - Process one argument at a time.

parallel (GNU Parallel)

Like xargs but distributes work across CPU cores. find . -name "*.gz" | parallel gunzip - Decompress files using all cores. Dramatically faster for I/O-bound operations.

Q43 ACLs - setfacl, getfacl

The Simple Answer

Standard Linux permissions (owner/group/others) are limited to three entities. ACLs (Access Control Lists) allow fine-grained permissions for multiple specific users and groups.

setfacl -m u:devuser:rw file - Give devuser read+write. setfacl -m g:devteam:rx dir - Give devteam group read+execute. getfacl file - View all ACLs. A + sign in ls -l output indicates ACLs are set.

Q44 trap and signal handling - cleanup on exit/error

The Simple Answer

trap defines actions to run when scripts receive signals or exit. Essential for cleanup - removing temp files, releasing locks, killing background processes.

Common Patterns

trap 'rm -f /tmp/lockfile' EXIT - Cleanup on exit (normal or error). trap 'echo "Interrupted"; exit 1' INT TERM - Handle Ctrl+C and kill. trap 'echo "Error on line \$LINENO"; exit 1' ERR - Catch any command failure.

Every production script should have a trap EXIT for cleanup. Without it, temp files accumulate, lock files persist, and background processes become orphans.

Q45 Error handling - set -euo pipefail

The Simple Answer

By default, Bash scripts continue executing after errors. set -euo pipefail makes scripts fail fast on any error.

set -e - Exit on any command failure. Without it, a failing command is silently ignored and the script continues with potentially corrupt state.

set -u - Treat unset variables as errors. Without it, \$UNDEFINED_VAR silently expands to empty string, causing subtle bugs.

set -o pipefail - A pipeline fails if any command fails, not just the last one. Without it, cat missing_file | grep pattern returns 0 (grep's exit code) despite cat failing.

Start every production script with: #!/usr/bin/env bash and set -euo pipefail. This catches 80% of scripting bugs at the point of failure instead of letting them cascade.

Q46 Here documents, here strings, template generation

Here Documents

Multi-line string input: `cat <<EOF` generates text until EOF. Variables are expanded inside. Use `<<'EOF'` (quoted) to prevent expansion. Essential for generating config files from scripts: `cat <<EOF > /etc/nginx/conf.d/app.conf server { listen 80; server_name $DOMAIN; } EOF.`

Here Strings

Single-line input: `grep "pattern" <<< "$variable"` - Feeds a variable as input to a command without echo and pipe.

Q47 File descriptors and ulimits

The Simple Answer

Every open file, socket, and pipe uses a file descriptor (a number). The default limit is 1024 per process. Production applications (databases, web servers) easily exceed this, causing "too many open files" errors.

Checking and Setting Limits

`ulimit -n` - Current file descriptor limit. `ulimit -n 65536` - Set for current session. `/etc/security/limits.conf` - Permanent per-user limits. For systemd services: `LimitNOFILE=65536` in the unit file.

The classic production incident: A Java application starts throwing "Too many open files" after running for days. The default 1024 limit is exhausted by database connections, HTTP connections, and log files. Fix: increase `LimitNOFILE` in the systemd unit file and restart.

Q48 fstab and mount options - noexec, nosuid, nodev

The Simple Answer

`/etc/fstab` defines filesystems mounted at boot. Mount options control security and behavior.

Security-Critical Options

noexec - Prevent executing binaries from this filesystem. Apply to `/tmp` to prevent attackers from running uploaded scripts.

nosuid - Ignore SUID/SGID bits. Prevents privilege escalation via uploaded SUID binaries.

nodev - Ignore device files. Prevents creating block/character devices for privilege escalation.

ro - Read-only. Good for boot partitions and static data.

Security best practice: Mount `/tmp` with `noexec,nosuid,nodev`. Mount `/var/tmp` similarly. These options prevent entire classes of attacks where attackers upload and execute malicious binaries through temporary directories.

Q49 File locking - flock

The Simple Answer

flock provides advisory file locking to prevent concurrent script execution. `flock -n /tmp/myscript.lock -c '/path/to/script.sh'` runs the script only if no other instance holds the lock. `-n` (non-blocking) exits immediately if locked. Without `-n`, it waits for the lock.

Why It Matters

Cron jobs that take longer than their interval can overlap. A backup script running every hour that takes 90 minutes overlaps with itself. flock prevents the overlap - the second instance skips or waits.

Q50 Process management from scripts - background, wait, job control

Background Execution

command & runs in background. `$_` stores the PID of the last background process. `wait $PID` waits for a specific background process to finish. `wait` (no argument) waits for all background processes.

Parallel Execution Pattern

Start multiple tasks in background, then wait for all: `task1 & task2 & task3 & wait` - All three run in parallel. The script continues after all finish.

Q51 Bash debugging - set -x, PS4, shellcheck

The Simple Answer

set -x - Print each command before execution. Shows exactly what Bash is doing. Toggle with `set +x`.
bash -x script.sh - Run entire script with tracing. **PS4** - Customize the trace prefix.
`PS4='+${BASH_SOURCE}:${LINENO}: '` shows filename and line number in trace output.

shellcheck

Static analysis tool for shell scripts. Catches common bugs: unquoted variables, incorrect comparisons, deprecated syntax, useless cat commands. Run in CI on every commit: `shellcheck script.sh`. Install with `apt install shellcheck`.

shellcheck is the single most impactful tool for shell script quality. It catches bugs that even experienced scripters miss. Add it to every CI pipeline that builds shell scripts.

Q52 Backup strategies - rsync, dd, incremental tar, snapshots

rsync - The Standard

`rsync -avz --delete /source/ /backup/` - Incremental sync. Only transfers changed bytes. `--delete` removes files in backup that were deleted from source. Use `--dry-run` first to preview changes. Works locally and over SSH.

dd - Block-level copy

`dd if=/dev/sda of=/backup/disk.img bs=4M` - Bit-for-bit disk copy. Used for full disk backups and forensics. Slow but complete - captures everything including deleted files and boot sectors.

LVM Snapshots

Create a point-in-time snapshot of a volume. Back up from the snapshot while the original continues serving traffic. Minimal impact on production. The gold standard for database backups on LVM-managed storage.

Q53 Systemd targets (run levels)

The Simple Answer

Systemd targets replaced traditional SysVinit run levels. Targets define system states.

multi-user.target - Normal multi-user system without GUI (run level 3). Most servers run here.

graphical.target - Multi-user with GUI (run level 5). Desktop systems.

rescue.target - Single-user mode with basic services (run level 1). For repair and recovery.

emergency.target - Minimal environment. Root shell only. Filesystem may not be mounted. Last resort for broken systems.

Set default: `systemctl set-default multi-user.target`. Switch: `systemctl isolate rescue.target`.

Q54 Network bonding/teaming

Network bonding combines multiple network interfaces for redundancy or performance. **Mode 1 (active-backup)** - One active, others standby. Failover on link failure. Most common for servers. **Mode 4 (802.3ad / LACP)** - Link aggregation. All interfaces active. Requires switch support. Best performance.

Q55 Bash vs sh vs zsh vs dash - shell compatibility

The Simple Answer

sh - POSIX standard shell. Minimal features. Scripts using `#!/bin/sh` should only use POSIX features. On Ubuntu, `/bin/sh` is dash (not bash), which breaks bash-specific syntax.

bash - GNU Bourne Again Shell. The standard on most Linux systems. Adds arrays, `[[ ]]`, process substitution, string manipulation.

zsh - Advanced interactive shell. Popular for daily use (Oh My Zsh). Not standard for scripting.

dash - Minimal POSIX shell. Faster startup than bash. Default `/bin/sh` on Debian/Ubuntu.

The #1 compatibility bug: Script starts with `#!/bin/sh` but uses bash features (arrays, `[[ ]]`). Works on RHEL (where `/bin/sh` is bash) but breaks on Ubuntu (where `/bin/sh` is dash). Always use `#!/usr/bin/env bash` for bash scripts. Use `#!/bin/sh` only with POSIX-compatible syntax. See scenario Q104.

Q56 LVM - physical volumes, volume groups, logical volumes

The Simple Answer

LVM adds a layer between physical disks and filesystems, enabling dynamic resizing, snapshots, and spanning across multiple disks.

Physical Volume (PV) - A disk or partition. `pvcreeate /dev/sdb`. **Volume Group (VG)** - A pool of one or more PVs. `vgcreate datavg /dev/sdb /dev/sdc`. **Logical Volume (LV)** - A virtual partition carved from a VG. `lvcreate -L 50G -n data1v datavg`. Format and mount the LV normally.

Why LVM Matters

Without LVM, resizing a partition requires downtime and data migration. With LVM: `lvextend -L +20G /dev/datavg/data1v && resize2fs /dev/datavg/data1v` grows the volume online with zero downtime.

Q57 RAID levels - 0, 1, 5, 6, 10

The Simple Answer

RAID 0 - Striping. Performance. No redundancy. One disk failure = total data loss. Never for important data.

RAID 1 - Mirroring. Two identical copies. Can lose one disk. 50% storage efficiency.

RAID 5 - Striping with parity. Can lose one disk. Minimum 3 disks. 67-94% efficiency. Good for read-heavy workloads.

RAID 6 - Double parity. Can lose two disks. Minimum 4 disks. Better reliability than RAID 5.

RAID 10 - Mirrored stripes. Performance + redundancy. Minimum 4 disks. 50% efficiency. Best for databases and high-performance storage.

Q58 Swap space - sizing, swapon/swapoff, swappiness

The Simple Answer

Swap is disk space used as virtual memory when RAM is full. The kernel moves inactive pages to swap to free RAM for active processes.

Swappiness

vm.swappiness controls how aggressively the kernel uses swap. Default is 60. Production database servers: set to 1 or 10 (swap only when absolutely necessary - databases manage their own memory). Desktop: keep at 60.

Sizing

Traditional rule: 1-2x RAM. Modern servers with 64+ GB RAM: 4-8 GB swap is sufficient. Hibernation: swap must be at least equal to RAM.

When swap is bad: If a production server is actively swapping, performance is terrible. Swap is a safety net, not a feature. If the server regularly uses swap, it needs more RAM or the application has a memory leak.

SECTION 3 - Senior Advanced Linux (Q59-Q80)

Q59 Linux kernel - modules, sysctl, /proc tuning

The Simple Answer

The kernel is the core of Linux - it manages hardware, processes, memory, and I/O. Kernel modules are loadable extensions (drivers, filesystems) that can be added without rebooting. sysctl tunes kernel parameters at runtime.

Kernel Modules

lsmod - List loaded modules. modprobe module_name - Load a module. modprobe -r module_name - Remove. /etc/modprobe.d/ - Persistent module configuration. /etc/modules-load.d/ - Modules to load at boot.

sysctl - Runtime Kernel Tuning

sysctl -a - List all parameters. sysctl net.ipv4.ip_forward=1 - Enable IP forwarding at runtime. /etc/sysctl.conf or /etc/sysctl.d/*.conf - Persistent settings. sysctl -p - Reload settings.

Common Production Tuning

net.core.somaxconn = 65535 - Maximum connection queue. vm.swappiness = 10 - Reduce swap usage. net.ipv4.tcp_max_syn_backlog = 65535 - Handle SYN floods. fs.file-max = 2097152 - Maximum open files system-wide. net.core.rmem_max = 16777216 - Socket receive buffer.

Q60 Linux capabilities - fine-grained privileges

The Simple Answer

Instead of giving a process full root or no root, capabilities split root privileges into 40+ individual permissions. A web server needs only CAP_NET_BIND_SERVICE (bind to port < 1024) - not full root.

How Capabilities Work

setcap cap_net_bind_service+ep /usr/bin/myapp - Allow binding to privileged ports without root. getcap /usr/bin/myapp - Check capabilities. Capabilities are set on the binary file, not the user.

Relevance to Containers

Docker drops most capabilities by default. You can add specific ones: docker run --cap-add NET_ADMIN - adds network administration. docker run --cap-drop ALL --cap-add NET_BIND_SERVICE - drop everything, add only what is needed. This is the principle of least privilege.

Q61 Namespaces and cgroups - the foundation of containers

Namespaces - Isolation

Each namespace type creates an isolated view: **PID** - Own process tree starting from PID 1. **Network** - Own IP addresses, routing table, ports. **Mount** - Own filesystem mount points. **User** - Own UID/GID mapping. **UTS** - Own hostname. **IPC** - Own inter-process communication.

cgroups - Resource Control

Control groups limit resource usage: **memory** - Hard memory limit. OOM kill if exceeded. **cpu** - CPU time allocation. **blkio** - Disk I/O throttling. **pids** - Maximum number of processes.

How They Create Containers

unshare creates processes in new namespaces. nsenter enters existing namespaces. A container is fundamentally: unshare --pid --net --mount --uts combined with cgroup resource limits and a union filesystem for the image layers. Docker automates this, but there is no "container" kernel feature - it is namespaces + cgroups + filesystem.

Q62 SELinux and AppArmor - mandatory access control

SELinux

Labels every file, process, port, and user with a security context. Policies define which labeled processes can access which labeled resources. Three modes: Enforcing (blocks violations), Permissive (logs but allows), Disabled.

Common commands: getenforce - Check mode. setenforce 0 - Switch to permissive. sestatus - Detailed status. ls -Z - Show security contexts. restorecon -R /path - Restore default contexts.

AppArmor

Path-based profiles define what each program can access. Simpler than SELinux. Default on Ubuntu and SUSE. aa-status shows loaded profiles. aa-complain sets a profile to log-only mode.

The interview question: "You deployed an application and it works in development but gets permission denied in production. Standard file permissions are correct. What do you check?" SELinux or AppArmor. The security context on the files does not match what the application's policy expects. See scenario Q90.

Q63 Linux networking deep dive - routing, iptables, NAT, bridging

Routing

ip route show - Display routing table. ip route add 10.0.0.0/8 via 192.168.1.1 - Add a route. Every packet is matched against the routing table to determine the next hop. The default route (0.0.0.0/0) handles traffic with no specific match.

iptables Chains

INPUT - Traffic destined for this machine. **OUTPUT** - Traffic originating from this machine. **FORWARD** - Traffic passing through this machine (routing). Each chain has rules evaluated in order. First match wins. Default policy (ACCEPT or DROP) applies if no rule matches.

NAT

SNAT - Change source address (masquerading). Used for internet access from private networks. **DNAT** - Change destination address (port forwarding). Used for exposing internal services. `iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE` - Enable NAT.

Bridging

A bridge connects multiple network interfaces at layer 2. Docker creates a bridge (docker0) and connects container virtual interfaces to it. `brctl show` or `ip link show type bridge` lists bridges.

Q64 TCP/IP stack tuning - socket buffers, conntrack, TIME_WAIT

TIME_WAIT Accumulation

After a TCP connection closes, the socket stays in TIME_WAIT for 2×MSL (usually 60 seconds). High-traffic servers can accumulate thousands of TIME_WAIT sockets, exhausting port range. Tuning: `net.ipv4.tcp_tw_reuse = 1` allows reusing TIME_WAIT sockets. `net.ipv4.ip_local_port_range = 1024 65535` widens available ports.

Connection Tracking (conntrack)

The kernel tracks every network connection for NAT and firewalling. The conntrack table has a finite size (default often 65536). Exceeding it causes packet drops: "nf_conntrack: table full, dropping packet." Fix: `net.netfilter.nf_conntrack_max = 262144`. This affects Docker and Kubernetes networking significantly.

Q65 Storage performance - I/O schedulers, fio, disk latency

I/O Schedulers

mq-deadline - Default for traditional HDDs. Fair scheduling with deadlines. **none** - For SSDs/NVMe. No scheduling overhead - the drive handles it. **bfq** - Budget Fair Queueing. Good for interactive workloads.

Benchmarking with fio

`fio --name=test --ioengine=libaio --rw=randread --bs=4k --numjobs=4 --size=1G --runtime=30` - Random read benchmark. Key metrics: IOPS, latency (p99), bandwidth.

Q66 Memory management - OOM killer, page cache, hugepages

OOM Killer

When the system runs out of memory, the OOM killer selects and kills a process based on an OOM score. Each process has an `oom_score_adj` (-1000 to 1000). -1000 means never kill (protect critical processes). 1000 means kill first.

Page Cache

Linux uses free RAM as file cache. cached memory in `free -h` is not wasted - it is used for caching file I/O and will be released when applications need RAM. Do not alarm at "high memory usage" if most of it is cache.

Hugepages

Standard memory pages are 4 KB. Hugepages are 2 MB or 1 GB. Reduces TLB misses for applications with large memory footprints (databases, VMs). Configure with `vm.nr_hugepages` in `sysctl`.

Q67 Linux hardening - CIS benchmarks, automated scanning

CIS Benchmarks

The Center for Internet Security publishes detailed hardening guides for every Linux distribution. Covers: filesystem configuration, user authentication, network settings, logging, access control. Hundreds of specific recommendations.

Automated Scanning

Lynis - Open-source security auditing tool. Scans the system and reports hardening recommendations with a score. **lynis audit system**. **OpenSCAP** - NIST SCAP implementation. Checks against CIS, DISA STIG, and custom profiles. Produces compliance reports.

Q68 /proc and /sys - runtime kernel inspection

/proc

`/proc/cpuinfo` - CPU details. `/proc/meminfo` - Memory statistics. `/proc/PID/status` - Process details. `/proc/PID/fd/` - Open file descriptors. `/proc/PID/maps` - Memory mappings. `/proc/loadavg` - Load average. `/proc/net/tcp` - Active TCP connections.

/sys

`/sys/class/net/` - Network interfaces. `/sys/block/` - Block devices. `/sys/class/thermal/` - Temperature sensors. Many `/sys` entries are writable for runtime configuration.

Q69 Disk encryption - LUKS

The Simple Answer

LUKS (Linux Unified Key Setup) provides full disk encryption. Data at rest is encrypted. Without the passphrase or key file, the data is unreadable - even if the disk is physically stolen. `cryptsetup luksFormat /dev/sdb1` creates an encrypted volume. `cryptsetup luksOpen /dev/sdb1` decrypted opens it for use.

Q70 Systemd advanced - custom units, dependencies, resource limits

Custom Unit Files

Create `/etc/systemd/system/myapp.service` with [Unit], [Service], and [Install] sections. [Service] defines: ExecStart (command to run), Restart=always (auto-restart on failure), User/Group (run as non-root), Environment (env vars), LimitNOFILE (file descriptor limit), MemoryMax (memory limit via cgroup).

Dependencies

After= - Start after this unit. Requires= - Fail if this unit fails. Wants= - Soft dependency. BindsTo= - Tight coupling.

Q71 Infrastructure automation with shell

When Shell Is Appropriate

Simple provisioning scripts, bootstrapping before Ansible/Terraform runs, cloud-init user-data scripts, CI/CD pipeline steps, and small utility scripts that run on every server. Keep scripts under 200 lines. Beyond that, switch to Python or a proper configuration management tool.

Q72 Log aggregation and rotation - logrotate, rsyslog, journald

logrotate

Rotates, compresses, and removes old log files. Configured in `/etc/logrotate.conf` and `/etc/logrotate.d/`. Key directives: daily/weekly, rotate 7 (keep 7 old files), compress, missingok, notifempty, postrotate (restart service after rotation).

journald

Systemd's logging system. Stores logs in binary format with indexing. Configure in `/etc/systemd/journald.conf`: SystemMaxUse (max disk usage), MaxRetentionSec (max age).

Q73 Security hardening - SSH, fail2ban, auditd

SSH Hardening

PermitRootLogin no - Disable root SSH. PasswordAuthentication no - Keys only. Port 2222 - Non-standard port (security through obscurity, not a replacement for real security). AllowUsers deploy - Whitelist allowed users. MaxAuthTries 3 - Limit attempts.

fail2ban

Monitors logs for failed authentication attempts. Bans offending IPs via iptables after configurable thresholds. Essential for any internet-facing SSH server.

auditd

Linux Audit Framework. Tracks file access, system calls, user commands. auditctl -w /etc/shadow -p rwa watches shadow file for read/write/attribute changes. ausearch -k keyword searches audit logs.

Q74 Certificate management - openssl, CSR, Let's Encrypt

Generating Certificates

openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout key.pem -out cert.pem - Self-signed certificate. openssl req -new -key key.pem -out request.csr - Certificate Signing Request for CA signing.

Checking Certificates

openssl x509 -in cert.pem -noout -dates - Check expiry. openssl s_client -connect host:443 - Test TLS connection and show certificate chain.

Let's Encrypt

Free, automated TLS certificates. certbot certonly --standalone -d example.com obtains a certificate. certbot renew --pre-hook "systemctl stop nginx" --post-hook "systemctl start nginx" automates renewal.

Q75 chroot - jail environments

chroot changes the apparent root directory for a process. The process can only see files within the chroot directory - it is "jailed." Used for: building packages in clean environments, recovering broken systems from a live CD, and running untrusted software with limited filesystem access. Chroot is not a security boundary - a root process can escape. Containers (namespaces + cgroups) replaced chroot for isolation.

Q76 NFS, SMB/CIFS - network file sharing

NFS

Linux-to-Linux file sharing. Server exports directories in `/etc/exports`. Clients mount with `mount -t nfs server:/path /mnt`. NFS v4 supports Kerberos authentication. Common in Kubernetes as a PersistentVolume backend.

SMB/CIFS

Windows file sharing protocol. Linux supports it via `samba` (server) and `cifs-utils` (client). `mount -t cifs //server/share /mnt -o username=user` mounts a Windows share.

Q77 Linux in cloud - cloud-init, user-data, metadata

cloud-init

Cloud-init runs during first boot of cloud instances. Reads configuration from user-data provided at launch. Sets hostname, creates users, installs packages, writes files, runs commands. The standard for bootstrapping cloud instances across AWS, GCP, Azure.

Metadata Service

Every cloud instance can query <http://169.254.169.254/> for instance metadata - instance ID, IP address, region, IAM role. Essential for self-configuring applications. Security: restrict access to the metadata service to prevent SSRF attacks.

Q78 Container runtime from scratch - unshare and nsenter

Building a Container Manually

`unshare --pid --net --mount --uts --fork /bin/bash` creates a process with isolated namespaces. Mount a root filesystem with `mount`. Set up a network interface. Apply cgroup limits by writing to `/sys/fs/cgroup/`. This is fundamentally what Docker does - packaged and automated.

nsenter

`nsenter` enters an existing namespace. `nsenter -t PID -n /bin/bash` enters the network namespace of PID. This is how docker `exec` works - it uses `nsenter` to enter the container's namespaces.

Q79 strace and ltrace - system call tracing

strace

Traces system calls made by a process. `strace -p PID` attaches to a running process. `strace -e open,read,write /path/to/command` traces specific syscalls. `strace -c command` shows a summary of syscall counts and times. Essential for debugging "why is this process hanging" - shows exactly what system call it is blocked on.

Practical Debugging

Application hangs? `strace -p PID` shows it is stuck on `futex()` (waiting for a lock) or `read()` (waiting for network data). Application fails silently? `strace` shows the exact system call that returned an error, like `open("/etc/missing.conf", O_RDONLY) = -1 ENOENT`.

Q80 Bash vs Python for DevOps - when to switch

Use Bash When

Short scripts (under 100 lines). Gluing CLI tools together. System administration tasks. CI/CD pipeline steps. Simple text processing. The task is primarily running commands and checking exit codes.

Switch to Python When

Script exceeds 200 lines. Complex data structures needed (nested dicts, classes). Error handling beyond simple exit codes. HTTP/API interaction. JSON/YAML processing beyond basic `jq`. Unit testing the script. Multiple developers maintaining it.

The practical rule: If you need more than two functions and an array, it is time for Python. If you are writing complex string parsing with `sed` and `awk` chained together, Python is clearer and more maintainable.

SECTION 4 - Tricky Scenarios & Real-World Disasters (Q81-Q112)

Q81 Server disk 100% full - find and fix without losing data

Step-by-Step Debugging

Step 1 - Confirm. `df -h` shows which filesystem is full. Note the mount point.

Step 2 - Find what is consuming space. `du -sh /* | sort -rh | head -10` shows the largest top-level directories. Drill down: `du -sh /var/* | sort -rh | head -10`. Continue until you find the culprit.

Step 3 - Common culprits. `/var/log` - Unrotated logs. `/var/cache` - Package manager cache. `/tmp` - Temp files. `/home` - User data. Docker: `/var/lib/docker` - Images, containers, volumes, build cache.

Step 4 - Quick wins. `apt clean` or `dnf clean all` - Clear package cache. `journalctl --vacuum-size=500M` - Reduce journal size. `find /tmp -type f -atime +7 -delete` - Delete old temp files. `docker system prune -a` - Clear unused Docker data.

Step 5 - Fix the root cause. Enable `logrotate`. Set journal size limits. Add disk monitoring with alerts at 80%.

Never blindly delete files in `/var` to free space. Deleting the wrong file can break system services. Always identify what is consuming space before deleting anything.

Q82 100% disk but du shows free space - deleted files held open

What Happened

`df -h` shows `/var` is 100% full. `du -sh /var` shows only 2 GB used. Where is the other 48 GB?

The Explanation

A large file was deleted (removed from the directory listing) but a process still has it open. The disk space is not released until the process closes the file descriptor. `du` cannot see the file because it is no longer in any directory. `df` sees the space as used because the file data still exists on disk.

Step-by-Step Fix

Step 1 - Find deleted files held open. `lsref +L1` lists files with zero links (deleted but held open). Look for large files: `lsref +L1 | grep deleted | sort -k7 -n`.

Step 2 - Identify the culprit. Usually a log file that was deleted while the application was still writing to it. The application keeps writing, the file keeps growing, but `ls` cannot see it.

Step 3 - Fix without restarting. Truncate the file descriptor: `> /proc/PID/fd/FD_NUMBER`. This empties the file without closing it. The application continues writing to a now-empty file.

Step 4 - Proper fix. Restart the offending process so it opens a new file. Fix log rotation to use copytruncate instead of create - this truncates the file in place rather than deleting and creating a new one, preventing the held-open problem.

This is one of the most common Linux production puzzles. Memorize the lsof +L1 command - it immediately reveals the hidden space consumption.

Q83 Cannot SSH into server - systematic debugging

Step-by-Step Debugging

Step 1 - Can you reach the server at all? ping server_ip. If no response: check if the server is up (cloud console), check network connectivity, check security groups/firewall allowing ICMP.

Step 2 - Is SSH port open? nc -zv server_ip 22 or nmap -p 22 server_ip. If closed: sshd might not be running, or firewall blocks port 22.

Step 3 - Is sshd running? If you have console access (cloud console, iDRAC, KVM): systemctl status sshd. If not running: systemctl start sshd. Check journalctl -u sshd for errors.

Step 4 - Can you connect but authentication fails? ssh -vvv user@server shows the authentication negotiation. Check: correct username, correct key file, key permissions (600 for private key, 700 for .ssh directory), authorized_keys has the public key.

Step 5 - Is the server rejecting connections? Check /etc/ssh/sshd_config for: AllowUsers (is your user listed?), DenyUsers (is your user blocked?), PasswordAuthentication (is it enabled if using password?), MaxAuthTries (too many failed attempts?).

Step 6 - Is it a firewall issue? Check iptables -L -n or ufw status on the server. Check cloud security groups. Check any intermediate firewalls or jump hosts.

Step 7 - Is it a DNS issue? If connecting by hostname: try the IP address directly. If IP works but hostname doesn't, it is a DNS problem.

The systematic approach matters in interviews. Do not guess - work through the layers: network → port → service → authentication → authorization. Each layer has specific diagnostic tools.

Q84 Running out of inodes but disk not full

What Happened

df -h shows plenty of free space. But creating new files fails with "No space left on device." df -i shows inodes are 100% used.

The Explanation

Every file and directory uses one inode. Millions of tiny files (mail queue, session files, cache entries) can exhaust inodes before filling disk space. Each filesystem has a fixed number of inodes set at creation time.

Step-by-Step Fix

Step 1 - Find the directory with millions of files. `find / -xdev -printf '%h\n' | sort | uniq -c | sort -rn | head -10` counts files per directory. This takes time but identifies the problem directory.

Step 2 - Common culprits. `/var/spool/mail` - Undelivered emails. `/tmp` - Session files from a web app. `/var/cache` - Package or application cache. `/var/lib/php/sessions` - PHP session files.

Step 3 - Delete the unnecessary files. `find /path -type f -mtime +7 -delete` removes files older than 7 days. For millions of files, `rm` is too slow - use `find` with `-delete`.

Step 4 - Fix root cause. Configure the application to clean up old files. Add a cron job for periodic cleanup. Set `tmpwatch` or `systemd-tmpfiles` to auto-clean temp directories.

Q85 Process consuming 100% CPU - identify and resolve

Step-by-Step

Step 1 - Identify the process. `top` or `htop` - sort by CPU (press P in top). Note the PID and command.

Step 2 - Determine if it is legitimate. Is it a known application during expected high load? A compile job? A legitimate workload? If so, it may just need time to complete.

Step 3 - If it is a runaway process. Check what it is doing: `strace -p PID` shows system calls. If it is in a tight loop, the application has a bug.

Step 4 - Reduce impact without killing. `renice -n 19 -p PID` lowers priority so other processes get CPU time. `ionice -c 3 -p PID` lowers I/O priority.

Step 5 - Kill if necessary. `kill PID` (SIGTERM first). Wait 10 seconds. `kill -9 PID` if still running.

Step 6 - Investigate root cause. Check application logs. Check for infinite loops, tight polling, unoptimized queries, or deadlocks. Profile the application with appropriate tools (`strace`, `perf`, language-specific profilers).

Q86 OOM killer killed production process

Step-by-Step

Step 1 - Confirm OOM kill. `dmesg | grep -i "oom\|killed process"` shows OOM messages with the killed process name and PID. `journalctl -k --since "1 hour ago" | grep -i oom` provides timestamped events.

Step 2 - Identify memory consumption. Check `/var/log/kern.log` or `dmesg` for the memory state at OOM time - total pages, free pages, and which process was chosen.

Step 3 - Determine the cause. Was the process legitimately using too much memory (needs more RAM)? Did it have a memory leak (memory usage grew over days)? Was another process consuming memory that pushed the victim over?

Step 4 - Fix. For legitimate memory needs: add RAM or increase the VM/container memory limit. For memory leaks: identify and fix the leak. For container environments: increase the container memory limit.

Step 5 - Protect critical processes. `echo -1000 > /proc/PID/oom_score_adj` protects a process from OOM killer (never kill). For systemd services: `OOMScoreAdjust=-1000` in the unit file.

Q87 Zombie processes accumulating

Step-by-Step

Step 1 - Find zombies. `ps aux | grep Z` shows zombie processes.

Step 2 - Find the parent. `ps -o ppid= -p ZOMBIE_PID` shows the parent PID. The parent is the problem - it is not calling `wait()` on its children.

Step 3 - Fix the parent. If the parent is a known application, fix its child process handling. If the parent is stuck, restart it. When the parent dies, zombies are adopted by `init/systemd` which reaps them.

Step 4 - You cannot kill a zombie. `kill -9` on a zombie does nothing - it is already dead. The only fix is addressing the parent process.

Q88 Cron job running but no output

Step-by-Step

Step 1 - Check if it is actually running. `grep CRON /var/log/syslog` or `journalctl -u cron` shows cron executing the job.

Step 2 - Check output redirection. If the cron entry has `> /dev/null 2>&1`, all output is discarded. Remove the redirection temporarily to see errors.

Step 3 - Check MAILTO. By default, cron emails output to the user. If no MTA is configured, output is silently lost. Set `MAILTO=""` to suppress or configure an MTA.

Step 4 - Add explicit logging. Redirect output to a file: `* * * * * /path/script.sh >> /var/log/myjob.log 2>&1`.

Q89 Script works manually but fails in cron

The Most Common Cause

Cron runs with a minimal environment. PATH is typically just /usr/bin:/bin. Commands that work in your interactive shell (because your PATH includes /usr/local/bin, /home/user/.local/bin, etc.) fail in cron because those directories are not in PATH.

Step-by-Step Fix

Step 1 - Use full paths. Replace python3 with /usr/bin/python3. Replace aws with /usr/local/bin/aws.

Step 2 - Set PATH in crontab. Add PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin at the top of the crontab.

Step 3 - Source the profile. Wrap the command: bash -l -c '/path/to/script.sh' runs a login shell that sources .bash_profile.

Step 4 - Check working directory. Cron's working directory is the user's home. If the script uses relative paths, they resolve differently than when you run it from the project directory. Use cd /path/to/project at the start of the script.

Step 5 - Check permissions and SELinux. The cron daemon runs as a different security context than your interactive shell. SELinux may block operations that work interactively.

Q90 Permission denied despite correct permissions - SELinux

Step-by-Step

Step 1 - Verify SELinux is the cause. Run the command, then check /var/log/audit/audit.log | grep denied. If you see AVC denials, SELinux is blocking.

Step 2 - Check security contexts. ls -Z file shows the SELinux context. The file's context must match what the application's policy expects. A file moved with mv preserves its old context. A file copied with cp gets the destination's context.

Step 3 - Fix the context. restorecon -R /path resets files to their default context based on the file path. This fixes most issues.

Step 4 - If restorecon does not help. The application may need a custom SELinux policy. Use audit2allow to generate a policy module from the denial logs: cat /var/log/audit/audit.log | grep denied | audit2allow -M mypolicy && semodule -i mypolicy.pp.

The shortcut that is never acceptable in production: setenforce 0 disables SELinux. Some engineers do this as a quick fix. This removes an entire security layer. Always fix the context or policy instead.

Q91 DNS resolution suddenly stopped

Step-by-Step

Step 1 - Test DNS. `dig google.com`. If it times out, DNS is broken. If it works with a specific server (`dig google.com @8.8.8.8`), the problem is the configured DNS server, not DNS itself.

Step 2 - Check `/etc/resolv.conf`. Is it empty? Has it been overwritten? Are the nameserver entries pointing to working DNS servers? On cloud instances, check if DHCP or NetworkManager has overwritten it.

Step 3 - Check `systemd-resolved`. On Ubuntu, DNS goes through `systemd-resolved`. `systemctl status systemd-resolved` shows its state. `resolvectl status` shows configured DNS servers per interface.

Step 4 - Check connectivity to DNS server. `ping NAMESERVER_IP`. If unreachable, it is a network issue, not DNS.

Step 5 - Check firewall. Is outbound UDP/TCP port 53 blocked? `iptables -L -n` or `ufw status`.

Q92 Server time drifting - NTP causing auth failures

Why Time Matters

Kerberos authentication allows 5-minute clock skew by default. TLS certificates are valid for specific time ranges. AWS API requests have 5-minute validity. Cloud IAM tokens are time-sensitive.

Step-by-Step Fix

Step 1 - Check current time. `date` shows system time. `timedatectl` shows time, timezone, and NTP status.

Step 2 - Check NTP. `timedatectl show-timesync` or `ntpq -p` shows NTP server status and offset. If NTP is not running: `systemctl start systemd-timesyncd` or `systemctl start ntpd`.

Step 3 - Force sync. `ntpdate pool.ntp.org` or `timedatectl set-ntp true`.

Step 4 - Verify NTP server accessibility. The server must reach NTP servers (UDP port 123). Check security groups and firewalls.

Q93 `/var/log` filling up disk - log rotation broken

Step-by-Step

Step 1 - Find the largest logs. `du -sh /var/log/* | sort -rh | head -10`.

Step 2 - Check `logrotate`. `cat /etc/logrotate.d/application` - Is the config correct? `logrotate -d /etc/logrotate.conf` - Dry run to check for errors.

Step 3 - Common issues. The log file path in `logrotate` config does not match the actual log file. The `postrotate` script fails (service restart command wrong). The log file is not created after rotation.

(application needs to be signaled to open a new file). copytruncate is not set and the application does not reopen the file after rotation.

Step 4 - Emergency cleanup. Truncate the log: `> /var/log/huge.log` empties it without deleting. Then fix rotation config.

Q94 SSH key auth stopped working after update

Common Causes After Updates

Permissions changed. SSH is strict: `~/.ssh` must be 700, `authorized_keys` must be 600, private key must be 600. Any deviation causes silent failure.

Deprecated key type. OpenSSH 8.8+ disabled RSA-SHA1 signatures by default. Old RSA keys may stop working. Fix: regenerate with ed25519 or add `PubkeyAcceptedAlgorithms +ssh-rsa` to `sshd_config`.

sshd_config overwritten. Package update replaced your config. Check for `.rpmnew` or `.dpkg-dist` files alongside the original.

SELinux context changed. The `authorized_keys` file has wrong SELinux context after update. `restorecon -R ~/.ssh`.

Q95 Filesystem went read-only

Why It Happens

The kernel detects filesystem errors and remounts as read-only to prevent corruption. Common causes: failing disk, bad sectors, cable issues, or filesystem corruption.

Step-by-Step

Step 1 - Check kernel messages. `dmesg | grep -i "error\|read.only\|EXT4"` shows the filesystem errors that triggered read-only mode.

Step 2 - Check disk health. `smartctl -a /dev/sda` shows SMART data. Look for Reallocated Sectors, Pending Sectors, and Uncorrectable Errors. If SMART shows failing: plan disk replacement immediately.

Step 3 - Fix the filesystem. If the disk is healthy, unmount and run `fsck`: `umount /dev/sda1 && fsck -y /dev/sda1`. For root filesystem: boot from a live CD/USB, then `fsck`.

Step 4 - Remount. `mount -o remount,rw /dev/sda1 /mountpoint`.

Q96 Network interface flapping

Check `dmesg` for link up/down messages. Common causes: bad network cable, failing NIC, switch port issues, duplex mismatch. Replace the cable first. If that does not fix it, check `ethtool eth0` for speed/duplex settings. Check the switch port. If the NIC is failing, replace it.

Q97 Fork bomb - system unresponsive

What Is a Fork Bomb

`:(){ :|:& };:` - A function that calls itself twice recursively, exponentially creating processes until the system runs out of PIDs.

Recovery

If you have an existing SSH session, try: `killall -9 -u bombed_user`. If no existing session works, use the console (cloud console, iDRAC). If completely locked: hard reboot.

Prevention

Set `ulimit -u` limits in `/etc/security/limits.conf`: `username hard nproc 1000`. This limits the user to 1000 processes. For systemd services: `TasksMax=512` in the unit file.

Q98 Kernel panic - check and recover

Step-by-Step

Step 1 - Reboot. Kernel panic is unrecoverable without reboot. If hung, hard reset.

Step 2 - After reboot, check logs. `journalctl --boot -1` shows logs from the previous boot. Look for the panic message and stack trace. `dmesg` may contain relevant hardware errors.

Step 3 - Common causes. Faulty hardware (RAM errors - run `memtest86`). Corrupt filesystem - run `fsck`. Bad kernel module - check if a recently loaded module caused it and blacklist it. Kernel bug - check if a recent kernel update caused it, roll back to previous kernel from GRUB.

Q99 Server compromised - incident response

Step-by-Step

Step 1 - Isolate. Disconnect from network. Do NOT shut down - volatile evidence (running processes, network connections, open files) is lost on shutdown.

Step 2 - Capture evidence. `ps auxf > /tmp/processes.txt`. `netstat -tlnp > /tmp/connections.txt`. `last > /tmp/logins.txt`. `cat /etc/passwd > /tmp/users.txt`. Take disk image with `dd` if possible.

Step 3 - Investigate. Check `/var/log/auth.log` for unauthorized logins. Check `crontab` for all users for backdoors. Check `/tmp` and `/dev/shm` for attacker tools. Check for unexpected SUID files: `find / -perm -4000`. Check for rootkits: `rkhunter`, `chkrootkit`.

Step 4 - Do not try to clean the server. Rebuild from known-good backup or fresh install. A compromised server can never be trusted again - the attacker may have installed backdoors you cannot find.

Step 5 - Post-incident. Rotate all credentials. Patch the vulnerability that was exploited. Review firewall rules. Update monitoring to detect similar attacks.

Q100 Server suddenly very slow - systematic load diagnosis

Step-by-Step

Step 1 - Check load average. uptime or cat /proc/loadavg. High load average indicates the system is overloaded. Compare to number of CPUs - load average equal to CPU count is 100% utilized.

Step 2 - Is it CPU? top - check %us (user) and %sy (system). If high, a process is consuming CPU. See Q85.

Step 3 - Is it memory? free -h - Is swap being used? If swap usage is high and growing, the system is thrashing. See Q107.

Step 4 - Is it I/O? iostat -x 1 - Check %util and await. If disk utilization is near 100% with high await (milliseconds), the disk is the bottleneck. See Q101.

Step 5 - Is it network? sar -n DEV 1 - Check network throughput. ss -s - Check for connection accumulation.

Step 6 - Narrow down. Once you identify the resource bottleneck (CPU, memory, I/O, network), find the process responsible using top, iotop, iftop, or ss and address it specifically.

The diagnostic sequence that impresses interviewers: uptime (load) → top (CPU) → free (memory) → iostat (disk) → ss (network). This systematic approach covers all possible bottlenecks in under 60 seconds.

Q101 High load average but CPU mostly idle - I/O wait

What Happened

Load average is 15 (on a 4-core server), but top shows CPU is 90% idle. The wa (I/O wait) column shows 80%.

Explanation

Load average counts processes in both runnable AND uninterruptible sleep (D state). Processes waiting for disk I/O are in D state. High I/O wait means processes are blocked waiting for slow disk.

Step-by-Step

Step 1 - Identify disk activity. iostat -x 1 shows per-device utilization and latency. iotop shows per-process I/O. The process with the most I/O is the likely culprit.

Step 2 - Common causes. Excessive logging. Database queries scanning large tables. Backup jobs running during peak hours. Swapping (memory pressure causing disk I/O). Multiple processes competing for the same disk.

Step 3 - Fix. Move heavy I/O to off-peak hours. Add faster storage (SSD/NVMe). Optimize database queries. Fix the memory issue causing swap. Separate I/O workloads to different disks.

Q102 Can't delete file even as root - immutable flag

What Happened

`rm -f /path/to/file` returns "Operation not permitted" even as root. `chmod` and `chown` also fail with the same error.

The Cause

The file has the immutable attribute set. This prevents ANY modification - even root cannot delete, rename, or modify the file.

Fix

`lsattr /path/to/file` shows the attributes. An 'i' flag means immutable. `chattr -i /path/to/file` removes the immutable flag. Now `rm` works normally.

Other Attributes

'a' (append-only) - Can only append, not overwrite or delete. Used for log files. 'e' (extent-based) - Normal for ext4. These attributes are separate from standard permissions and ACLs.

Q103 Application can't bind to port 80

Step-by-Step

Step 1 - Is something already using port 80? `ss -tlnp | grep :80` shows the process. If another service is bound, stop it or reconfigure.

Step 2 - Is the application running as non-root? Ports below 1024 require root or the `CAP_NET_BIND_SERVICE` capability. Fix: `setcap cap_net_bind_service+ep /path/to/binary` or run behind a reverse proxy (nginx on port 80 proxying to the app on port 8080).

Step 3 - Is SELinux blocking? SELinux restricts which ports services can bind to. `semanage port -l | grep http` shows allowed HTTP ports. `semanage port -a -t http_port_t -p tcp 8080` allows a new port.

Q104 Script works on one server, fails on another

Common Causes

Shell mismatch. Script uses `#!/bin/sh` but relies on Bash features (arrays, `[[ ]]`). On Ubuntu, `/bin/sh` is dash, which lacks these features. Fix: change shebang to `#!/usr/bin/env bash`.

Different PATH. A tool is installed in `/usr/local/bin` on one server but not the other, or the PATH is different. Fix: use full paths to commands.

Different tool versions. GNU vs BSD versions of `sed`, `grep`, `awk` behave differently (especially on macOS vs Linux). Fix: use POSIX-compatible syntax or check for version-specific features.

Different locale. `LC_ALL` affects sorting, regex, and string comparison. Fix: set `LC_ALL=C` in scripts for consistent behavior.

Q105 High TIME_WAIT connections - socket exhaustion

Step-by-Step

Step 1 - Confirm. `ss -s` shows TIME-WAIT count. `ss -tan | grep TIME-WAIT | wc -l` counts them.

Step 2 - Why it matters. Each TIME_WAIT holds a source port for 60 seconds. With limited port range (default 28232 ports), high connection rates can exhaust available ports. New connections fail with "Cannot assign requested address."

Step 3 - Fix. `sysctl net.ipv4.tcp_tw_reuse=1` - Allow reusing TIME_WAIT sockets. `sysctl net.ipv4.ip_local_port_range="1024 65535"` - Widen port range. Use connection pooling in the application to reduce connection creation rate.

Q106 SSH connects but drops after authentication

Common Causes

Shell not available. The user's shell (in `/etc/passwd`) does not exist or is not executable. Fix: `chsh -s /bin/bash` user.

Profile script error. `.bashrc` or `.bash_profile` has an error that causes the shell to exit. Test: `ssh user@server /bin/bash --norc --noprofile`. If this works, fix the profile script.

ForceCommand in sshd_config. An `sshd` config directive forces a specific command and exits. Check Match blocks in `sshd_config`.

PAM configuration. A PAM module is denying the session. Check `/var/log/auth.log` for PAM errors.

Resource limits. `ulimit` settings preventing the user from starting a shell. Check `/etc/security/limits.conf`.

Q107 Swap thrashing - system crawling

Step-by-Step

Step 1 - Confirm swapping. `vmstat 1` shows `si` (swap in) and `so` (swap out) columns. Sustained non-zero values mean active swapping. `free -h` shows swap usage.

Step 2 - Find the memory hog. top sorted by RES (resident memory). Or `ps aux --sort=-%mem | head`. The process using the most RSS is likely pushing others to swap.

Step 3 - Fix the immediate issue. Kill or restart the memory hog. Or add swap temporarily: `fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile`.

Step 4 - Fix root cause. Reduce `vm.swappiness` to 1 or 10 for servers. Add more RAM. Fix the memory leak in the application. Set memory limits in `systemd` to prevent a single process from consuming all RAM.

Q108 Accidentally ran `rm -rf` on wrong directory

Harsh Reality

On a standard Linux filesystem, deleted files are gone. There is no recycle bin. `rm -rf` is immediate and permanent.

Recovery Attempts

Step 1 - Stop writing to the disk immediately. Every write potentially overwrites the deleted data. Remount read-only if possible: `mount -o remount,ro /dev/sda1`.

Step 2 - Try recovery tools. `extundelete` (for ext3/ext4) or `photorec` can sometimes recover files. Success depends on how much new data has been written since deletion.

Step 3 - Restore from backup. This is the realistic recovery path. If you have no backups, the data is likely gone.

Prevention

Use `trash-cli` instead of `rm` for interactive use. Use `rm -i` or alias `rm='rm -i'` for interactive confirmation. For scripts, use `set -euo pipefail` to prevent variable expansion errors that cause wrong paths. Always test `rm` commands with `echo` first: `echo rm -rf $DIR` - verify the expanded path looks correct before removing the echo.

Q109 Accidentally ran `chmod -R 777 /`

What Breaks

Everything. SSH stops working (key files must be 600, `sshd` refuses insecure keys). `sudo` stops working (`sudoers` file must not be world-writable). SUID binaries stop working. Every security control is disabled.

Recovery

Step 1 - Do not reboot. Current sessions keep working. A reboot makes things worse because services fail to start with wrong permissions.

Step 2 - Fix critical permissions immediately from current session. `chmod 600 /etc/ssh/ssh_host_*` && `chmod 644 /etc/ssh/ssh_host_*.pub` (SSH host keys). `chmod 440 /etc/sudoers` (sudo). `chmod 4755 /usr/bin/sudo /usr/bin/passwd` (SUID). `chmod 755 /etc/var /usr/bin/sbin` (directories).

Step 3 - Restore all permissions. Reinstall all packages: `apt-get --reinstall install $(dpkg --get-selections | awk '{print $1}')` or `rpm --setperms $(rpm -qa)`. This restores file permissions to package defaults.

Step 4 - If the system is unsalvageable, rebuild from backup or fresh install.

Prevention: Never run `chmod -R` on `/` or any system directory. Use specific paths. If a script builds a path dynamically, add a safety check: `[[ "$DIR" == "/" ]] && echo "DANGER" && exit 1`.

Q110 Write a script: disk monitoring with alerting

The Script Requirements

Check disk usage on all mounted filesystems. Alert (log + email/Slack) if any filesystem exceeds 80%. Run via cron every 5 minutes.

Key Implementation Points

Use `df -h` and `awk` to extract usage percentages. Compare against threshold. Send alert via `curl` to a Slack webhook or mail command. Include hostname, filesystem, usage percentage, and available space in the alert. Use `flock` to prevent overlapping runs. Use `set -euo pipefail` for safety.

The Production-Grade Additions

Suppress repeated alerts for the same filesystem (track state in a file). Different thresholds for warning (80%) and critical (90%). Include top 10 space consumers from `du` in the alert for immediate diagnosis.

Q111 Parse 50GB log file - top 10 IPs and attack patterns

The Challenge

The file is too large for a text editor. Loading it into memory is impractical. You need streaming tools.

Step-by-Step

Step 1 - Top 10 IPs by request count. `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10`. This streams through the file without loading it into memory. For a 50 GB file, this takes 5-15 minutes.

Step 2 - Find attack patterns. `grep -c "\.\/" access.log` - Directory traversal attempts. `grep -ci "union.*select\|drop.*table" access.log` - SQL injection attempts. `grep -c "eval\|exec\|system" access.log` - Code injection attempts.

Step 3 - Filter by time range. `awk '/01\/Jun\/2025:10:/ {print $1}' access.log | sort | uniq -c | sort -rn` - Requests in a specific hour.

Step 4 - For repeated analysis. If you need to query the log multiple times, consider loading it into a tool designed for large files: GoAccess for web logs, or pipe into SQLite for ad-hoc queries.

The key principle: Use streaming tools (awk, grep, sort, uniq) that process files line by line without loading them into memory. Never `cat file | grep` - use `grep file` directly (avoids a useless subprocess).

Q112 Write a deployment script with rollback

Key Design Principles

Idempotent - Running the script twice produces the same result. Use create-or-update patterns, not create-and-fail.

Rollback capability - Before deploying, save the current state (image tag, config version). If deployment fails, restore the saved state.

Health verification - After deployment, check if the application is healthy. If health checks fail within a timeout, trigger automatic rollback.

Logging - Log every action with timestamps. Log the current and target versions. Log the rollback trigger reason.

The Script Flow

Parse arguments (getopts). Validate inputs. Save current state. Deploy new version. Wait for health checks (with timeout). If healthy: log success. If unhealthy: trigger rollback to saved state, log failure, exit with error. Send notification (Slack/email) with result.

Error Handling

Use `set -euo pipefail`. Wrap deployment and rollback in functions. Use `trap` to handle unexpected errors: `trap 'rollback; echo "Deployment failed"; exit 1' ERR`. This ensures rollback happens even on unexpected script errors.

Interview Tips

For Junior Roles

Focus on Q1-Q30. Know filesystem hierarchy, permissions, process management, and basic networking. Be able to use grep, find, and basic text processing. Know how to debug with logs and systemctl.

For Mid-Level Roles

Q1-Q58. Be ready to write a shell script on the spot. Know grep/sed/awk. Understand error handling (set -euo pipefail). Know LVM, RAID, and system tuning. Have a real story about a production Linux issue you debugged.

For Senior Roles

All 112 questions. Kernel tuning, namespaces/cgroups, TCP/IP stack tuning, and security hardening separate senior from mid. Be ready to discuss performance profiling with strace, container internals, and CIS benchmarks.

For Scenario Questions

Walk through systematically. State what you would check first, which command, what the output tells you, and your next step. The diagnostic sequence - uptime → top → free → iostat → ss - should be automatic.

Quick Reference Cheat Sheet

Topic	Key Takeaway
Permissions	755 dirs, 644 files, 600 secrets. Never 777. Always quote variables.
Process states	R=running, S=sleeping, D=I/O wait, Z=zombie, T=stopped.
Zombie fix	Kill the parent, not the zombie. Zombie is already dead.
Disk full	df -h then du -sh /* drilling down. Check lsof +L1 for deleted files.
Inode exhaustion	df -i to check. Find dir with millions of tiny files.
SSH debug	ssh -vvv. Check permissions (600/700). Check sshd_config. Check SELinux.
Cron fails	Full paths. Set PATH. Check environment. Redirect output to log.
set -euo pipefail	Every production script. Catches 80% of bugs at failure point.
grep/sed/awk	grep searches. sed transforms. awk extracts and computes.
Signals	SIGTERM first (graceful). SIGKILL last resort (no cleanup). SIGHUP reloads.
Load average	Compare to CPU count. High load + low CPU = I/O wait (iostat).
OOM killer	dmesg shows kill. Protect with oom_score_adj=-1000. Fix memory leak.
SELinux	restorecon fixes contexts. Never setenforce 0 in production.
Swap	Swappiness 1-10 for servers. Active swapping = need more RAM.
LVM	Resize online with lvextend + resize2fs. Snapshots for backups.
strace	Traces syscalls. Reveals what a hanging process is waiting for.

fstab security	noexec,nosuid,nodev on /tmp. Prevents execution attacks.
ulimits	Default 1024 file descriptors. Production needs 65536+.
Bash vs Python	Under 100 lines and gluing commands = Bash. Anything more = Python.
shellcheck	Static analysis for scripts. Run in CI. Catches bugs experts miss.
rm safety	echo the command first. Use trash-cli. set -euo pipefail prevents variable mishaps.
Log rotation	Configure logrotate. Use copytruncate for apps that don't reopen files.